1. Lab brief

You are building a prototype to estimate the time required to resolve software support tickets. Use a supplied synthetic dataset to compare linear regression, polynomial regression, Ridge, and Lasso. The goal is not to claim the synthetic data represents a real organization; the goal is to reason about model form, error metrics, regularization, and a reliable evaluation workflow.

2. Learning objectives

- Load and inspect a tabular regression dataset.
- Create a reproducible train/test split and a validation strategy.
- Fit a baseline and compare multiple regression models.
- Report MAE, RMSE, and R^2 and inspect residuals.
- Explain how polynomial degree and regularization affect fit and generalization.
- Identify at least two limitations or risks before deployment.

3. Dataset description

Column	Meaning	Type
ticket_id	Synthetic unique row identifier; do not use as a predictive feature.	Identifier
complexity_score	Estimated issue complexity, on a 1–10 scale.	Numeric feature
lines_changed	Number of code lines changed during the fix; check whether this would be available at prediction time.	Numeric feature / possible leakage risk
team_experience_years	Approximate team experience for the ticket.	Numeric feature
reopened	Whether the ticket was reopened (0/1).	Binary feature
resolution_hours	Observed resolution time; the target to predict.	Numeric target

4. Tasks

1. Load the CSV. Display shape, data types, descriptive statistics, missing values, and the first five rows.
2. Define `resolution_hours` as y. Exclude `ticket_id` from X. Write down which features are available at the intended prediction time and justify exclusions.
3. Split data into training and test sets (suggested 80/20, random_state=42). Do not use the test set to choose model settings.
4. Fit a `DummyRegressor(strategy='mean')` baseline and record its test metrics.
5. Fit `LinearRegression` on the selected features. Record MAE, RMSE, and R^2.
6. Create a pipeline with `PolynomialFeatures` and `StandardScaler` followed by linear regression. Compare degrees 2 and 3 using cross-validation on the training set.
7. Fit Ridge and Lasso models using a pipeline that scales features. Select alpha with cross-validation or a small validation grid. Record the selected value and results.

8. Plot observed versus predicted values and a residual plot for at least two models. Look for curvature, changing spread, and influential outliers.
9. Write a short recommendation: which model would you choose for this dataset, what evidence supports it, and what further data or checks would be needed?

5. Starter code

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, KFold, cross_val_score
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

df = pd.read_csv("lab_ticket_resolution.csv")
print(df.shape)
print(df.dtypes)
print(df.describe(include="all"))
print(df.isna().sum())
```

7. Analysis questions

10. Why is the mean baseline useful even though it ignores all features?
11. Which model has the lowest MAE? Which has the lowest RMSE? Are they the same, and why might they differ?
12. Did polynomial features improve validation performance? What is the evidence of possible overfitting?
13. How did regularization affect performance and coefficient magnitudes?
14. Why is scaling included in the Ridge/Lasso pipeline?
15. Identify one potential leakage issue in the dataset and explain how you handled it.
16. Would a random split be appropriate if tickets were collected over several years or came from repeated customers/teams? Explain.
17. What additional tests would you perform before deploying this model?